

前後端部署流程

本文件說明 **SAD-frontend-v2** 的前端、後端與 PostgreSQL 部署流程。專案目前採用：

- Frontend: React + Vite，建置後輸出到 **dist/**
- Backend: Node.js HTTP server，入口為 **backend/server.mjs**
- Database: PostgreSQL
- API base path: **/api/v1**
- Backend cloud target: Render Web Service + Render PostgreSQL
- Frontend cloud target: Vercel Static Frontend

部署架構

```
Browser
  |
  | Frontend static files
  v
Vercel
  |
  | VITE_API_BASE_URL
  v
Render Web Service
  |
  | DATABASE_URL
  v
Render PostgreSQL
```

前端不直接連資料庫，所有資料都透過 Render 後端 API 存取。

本機部署前檢查

從專案根目錄執行：

```
npm install
npm run lint
npm run build
```

若要同時驗證本機 API 與前端：

```
npm run db:setup
npm run api:dev
```

另開一個 terminal：

```
npm run dev
```

本機預設服務：

```
Frontend: http://127.0.0.1:5173
Backend:  http://127.0.0.1:8787/api/v1
Health:   http://127.0.0.1:8787/api/v1/health
```

`vite.config.ts` 已設定開發 proxy：

```
/api/v1 -> http://127.0.0.1:8787
```

所以本機開發時前端可直接呼叫 `/api/v1`。

後端部署：Render

Render 部署設定在 `render.yaml`：

```
services:
  - type: web
    name: sad-frontend-v2-api
    runtime: node
    plan: free
    buildCommand: npm ci && npm run db:init:cloud
    startCommand: npm run api:start
    healthCheckPath: /api/v1/health
    envVars:
      - key: HOST
        value: 0.0.0.0
      - key: DATABASE_URL
        fromDatabase:
          name: sad-frontend-v2-db
          property: connectionString

databases:
  - name: sad-frontend-v2-db
    plan: free
```

部署步驟：

1. 將 repo 推到 GitHub。
2. 在 Render 建立 Blueprint，選擇此 repo。
3. Render 讀取 `render.yaml` 後會建立：
 - `sad-frontend-v2-api`
 - `sad-frontend-v2-db`
4. Render build 階段執行：

```
npm ci
npm run db:init:cloud
```

5. Render start 階段執行：

```
npm run api:start
```

6. 部署完成後檢查：

```
https://<render-service-url>/api/v1/health
```

預期回應：

```
{
  "data": {
    "ok": true
  }
}
```

後端環境變數

Render 目前需要：

Key	用途	範例
HOST	讓 Render 可從 container 外部連入	0.0.0.0
PORT	Render 會自動注入，程式會讀取	Render managed
DATABASE_URL	PostgreSQL connection string	Render database connection string

backend/server.mjs 預設：

```
PORT=8787
HOST=127.0.0.1
```

在雲端必須使用 **HOST=0.0.0.0**。

資料庫初始化

本機完整重建：

```
npm run db:setup
```

雲端 schema 初始化：

```
npm run db:init:cloud
```

相關 scripts：

Script	實際指令	用途
db:setup	node backend/scripts/setupDatabase.mjs	本機 drop/create database，套 migration 和 seed
db:setup:cloud	DATABASE_SETUP_MODE=schema node backend/scripts/setupDatabase.mjs	雲端重建 public schema
db:init:cloud	DATABASE_SETUP_MODE=schema-if-empty node backend/scripts/setupDatabase.mjs	雲端初次初始化 schema

目前 migrations 會依序套用：

```
001_core_schema.sql
002_burden_tasks_schema.sql
003_allocation_schema.sql
004_stat_orders_schema.sql
005_handoff_schema.sql
006_allocation_decision_logs.sql
```

接著套用 demo seeds，並建立第一班預分配資料。

重要注意

目前 `db:init:cloud` 的空資料庫判斷在 `backend/scripts/setupDatabase.mjs`：

```
select to_regclass('public.units') as marker
```

但目前 schema 沒有 `units` table。這代表現行設定比較適合 demo 環境；正式部署前建議把 `marker` 改成實際存在的核心資料表，例如：

```
select to_regclass('public.users') as marker
```

否則重新部署時可能重新建立 schema 與 demo data。

前端部署：Vercel

前端為 Vite SPA，建置輸出在 `dist/`。

Vercel 設定在 `vercel.json`：

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

這讓 React Router 的頁面重新整理時會回到 `index.html`，避免 404。

Vercel 專案設定：

設定	值
Framework Preset	Vite
Build Command	<code>npm run build</code>
Output Directory	<code>dist</code>
Install Command	<code>npm ci</code>

前端環境變數：

Key	用途	範例
<code>VITE_API_BASE_URL</code>	前端 API base URL	<code>https://<render-service-url>/api/v1</code>

`src/api/client.ts` 的預設值是：

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? '/api/v1'
```

因此部署到 Vercel 時必須設定 `VITE_API_BASE_URL`，讓前端呼叫 Render API。

更新部署流程

一般更新：

1. 本機修改程式。
2. 執行：

```
npm run lint
npm run build
```

3. Commit 並 push 到 GitHub。
4. Render 自動重新部署後端。
5. Vercel 自動重新部署前端。
6. 檢查後端 health endpoint。

7. 開啟前端，確認主要流程可用：

- 目前班別
- 我的病人
- 麻煩度評估
- 任務狀態更新
- 分床建議
- 交班表

常見問題檢查

前端畫面出現 API 失敗

檢查 Vercel 的 `VITE_API_BASE_URL` 是否指向 Render：

```
https://<render-service-url>/api/v1
```

不要只填 Render root URL，也不要漏掉 `/api/v1`。

Render health check 失敗

檢查：

- `HOST=0.0.0.0`
- `DATABASE_URL` 是否存在
- Render PostgreSQL 是否建立成功
- build log 中 `npm run db:init:cloud` 是否成功

資料重新變回 demo 初始狀態

檢查 `db:init:cloud` 的 marker table。正式環境應避免每次 deploy 都 drop/recreate schema。

CORS 問題

後端 `backend/server.mjs` 會設定 CORS headers，前端部署到 Vercel 後可跨域呼叫 Render API。若仍發生問題，先確認瀏覽器 Network tab 的 request URL 是否為正確的 Render API URL。

部署檔案索引

檔案	用途
<code>render.yaml</code>	Render 後端與 PostgreSQL Blueprint
<code>vercel.json</code>	Vercel SPA fallback rewrite
<code>package.json</code>	前端 build、後端 start、DB setup scripts
<code>vite.config.ts</code>	本機開發 proxy
<code>backend/server.mjs</code>	API server entrypoint
<code>backend/scripts/setupDatabase.mjs</code>	migration / seed / cloud init

檔案

用途

src/api/client.ts

前端 API base URL 設定